

AMIR'S LAB COMPANION

Table of Contents

AMIR'S LAB COMPANION

A Step-by-Step Guide to Practical Work 1-5

Companion Volume to *Seeing Like a Machine* — DEC50302 Machine Learning in Computer Vision

Prepared for the Department of Electrical Engineering, Politeknik Sultan Abdul Halim Mu'adzam Shah (POLIMAS)

How to Use This Companion

This is the second half of a two-book set. *Seeing Like a Machine* teaches the ideas — why an image is a matrix, why HSV beats BGR for tracking, why a loss function exists. This book, **Amir's Lab Companion**, is where you actually sit down at a keyboard and build the five official Practical Works of DEC50302, in the exact order your lecturer assesses them.

Every task, every line of code, and every objective in this companion matches your official Practical Work sheets (PW1–PW5) **exactly** — nothing has been renamed or reordered, so what you submit on CIDOS will match what you practised here. Where the original code had a rough edge — a missing safety check, a hardcoded path that only works on one lecturer's laptop — this companion fixes it quietly and explains why, so your version is a little more robust than the one you started with. Nothing about the *task* changes; only the *reliability* of the code improves.

Three boxes will guide you through each lab:

 **Fixed in this Companion** — marks a small robustness improvement made to the original code (a missing `None` check, a path that now works on any machine, a friendlier error message). The task and the expected result are unchanged.

 **Common Mistake** — a mistake almost every student makes at this exact step, and how to recognise it fast.

 **Concept Bridge** — points you back to the exact section of *Seeing Like a Machine* where the underlying theory is explained in full, in case the “why” isn't clicking yet.

Amir, the POLIMAS student building a Smart Guardhouse throughout *Seeing Like a Machine*, walks through these same five labs in the same order you will. His guardhouse project is, in fact, built almost entirely out of PW1 through PW5 — by the time you finish PW5, you will have built every core component he needed.

Before every lab: use the computer lab only under supervision, with clean hands and no food or drinks. Handle equipment carefully, log in and out properly, save and back up your work as you go, and use only authorised software and websites. Keep your workstation ergonomic, report any equipment issues immediately, avoid electrical hazards, and know your lab's emergency procedures. Respect the privacy and access rights of the people around you.

After every lab: your report should always contain four sections — **Results** (every script's code with comments, plus the screenshots or output files it produced), **Discussion** (what you observed, what surprised you, what went wrong and why), **Conclusion** (whether your results supported what the lab set out to show, what likely caused any errors, and what you personally learned), and the code uploaded to CIDOS. This structure is graded against the Data Acquisition and Communication rubric (Results/Analysis, Question/Discussion, and Conclusion, each out of 10 marks, worth 30% of your total CA), on top of the Practical Skill rubric (worth 70%) that grades whether your code runs correctly without assistance. Write your discussion and conclusion in your own words — a rubric rewards evidence of thinking, not a restatement of the instructions.

PRACTICAL WORK 1 — Introduction to Python Programming

CLO 2 · Construct suitable application using machine learning method · **PL04, P4** · DK3, DK4, DK8 / DP1, DP4

Amir's Notebook

Before Amir could build anything, he needed a place to build it. PW1 is that place — not glamorous, but it is the single lab every other lab in this book depends on. Skip a step here and PW2 through PW5 will punish you for it later.

A. Objectives

1. Install Python and configure the Visual Studio Code environment correctly.
2. Create a project file and write a Python script using VS Code correctly.
3. Construct a given task and perform compiling and testing successfully.

B. Learning Outcome

Upon completion, you should be able to install Python and configure VS Code correctly, and create a project file and write a Python script using VS Code correctly.

 **Concept Bridge** — *Seeing Like a Machine*, Unit 1.1 (“The Python Paradigm”) explains *why* Python’s interpreted, line-by-line execution model makes it the right tool for this course before you install a single byte of it.

C. Topic Summary — Foundations of Python Programming

Python has become the *lingua franca* of machine learning and computer vision because of its syntactical clarity, its enormous ecosystem of scientific libraries, and its interoperability with C++-based engines like OpenCV. Mastering Python here is not about memorising syntax — it is about learning to think computationally.

Python is an interpreted, high-level, general-purpose language. Unlike compiled languages such as C or Java, which translate the entire program to machine code before anything runs, Python executes line by line through an interpreter. This matters for computer vision work specifically: you can tweak a threshold value and see the new result immediately, without waiting for a recompile.

Python’s design also favours readability through significant indentation rather than curly braces — the visual shape of your code mirrors its logical shape, which matters once your loops and conditionals start nesting three or four levels deep while processing image regions.

Setting up the environment. The syllabus recommends IDEs such as PyCharm, VS Code, Spyder, or Atom. This course standardises on **Visual Studio Code**, since it is versatile enough to combine multiple languages in a single workspace. Full step-by-step installation guidance:

<https://code.visualstudio.com/docs/python/python-tutorial>.

Setup has three parts: installing the **Python interpreter** (latest stable Python 3), **package management** via `pip install opencv-python numpy tensorflow`, and, as best practice, isolating each project with a **virtual environment** (`venv` or `conda`).

After installation, sign in to Microsoft’s free introductory Python course at <https://vscodeedu.com/courses/intro-to-python> using your polytechnic ID (`id@student.polimas.edu.my`). Polytechnic students get free access under Microsoft Education.

Core programming elements you will lean on for the rest of the semester: dynamic typing (a variable holds whatever you last assigned it — an `int` one moment, a `float` the next), operator precedence (PEMDAS — critical for image transformation equations), control flow (`if/elif/else`, `for`, and the `while True:` loop that is the standard skeleton for every video-processing script you will write), the four built-in data structures (List, Tuple, Dictionary, Set), functions (`def process_image(img):`), and Object-Oriented Programming — where a *class* is a blueprint and an *object* is the specific house built from it, complete with public/protected/private access levels, inheritance, polymorphism, and magic methods like `__str__`.

D. Material / Tools

1. PC installed with Visual Studio Code

2. CIDOS Online Platform

E. General Instruction / Safety Procedure

See “Before every lab” in the introduction to this companion.

F. Work Instruction / Procedure

PART A — Installing the Python Interpreter

1. On Windows, download Python from python.org/downloads and select the standalone installer for the current Python 3.x release.
2. Run the installer. **Tick “Add python.exe to PATH”** before clicking *Install Now* — this single checkbox saves you from step 8 below entirely. If you missed it, don’t worry, it is recoverable.
3. Follow the installer through to completion; you should see a “Setup was successful” confirmation.
4. Close any Command Prompt windows that were already open — the PATH variable only updates for *new* terminal sessions.
5. **(Optional) Set up PATH manually**, only if you missed the checkbox in step 2: search “envir” in the Windows search box → *Edit the system environment variables* → *Environment Variables...* → select the Path variable → *Edit* → *New* → add both:

Python

```
C:\Users\<username>\AppData\Local\Programs\Python\Python3x  
C:\Users\<username>\AppData\Local\Programs\Python\Python3x\Scripts
```

Click OK on every open dialog to save.

6. **Verify the installation.** Open a new Command Prompt and type:

Bash

```
python
```

You should land on Python’s >>> interactive prompt.

7. **Run your first line of Python:**

Python

```
print('Hello World')
```

Python is case-sensitive — `print` and `Print` are different things — and the quotes around `Hello World` are what make it a string.

8. **Exit the interpreter** with `exit()` or Ctrl-Z then Enter.

⚠ Common Mistake — Typing `Print('Hello World')` with a capital P. Python will tell you `Print` is not defined. Case sensitivity trips up almost every student exactly once — better it happens here than inside a 40-line detector script in PW3.

PART B — Installing the Visual Studio Code IDE

1. Download VS Code from code.visualstudio.com for your OS.
2. Run the installer and follow the prompts through to completion.
3. Open VS Code's own *Get Started* tutorial (Help menu, or the built-in walkthrough) to explore the interface: the file explorer, the integrated terminal, IntelliSense code completion, and how to install extensions (you will want the official **Python** extension from Microsoft).

PART C — Guided Tasks

Task 1 — Thresholding logic simulation. This connects the abstract idea of a loop to the concrete computer vision task of **thresholding**. When you later call `cv2.threshold()` in PW3, you will already understand the logic running underneath it.

Python

```
# Micro-Exercise 1.2: Thresholding Logic Simulation
# Objective: Use a for-loop to apply a binary threshold
logic.

# A list simulating a single row of grayscale pixels
image_row = [20, 150, 45, 200, 10, 127, 180, 90]
threshold_value = 127
binary_row = []

print("Original Scanline:", image_row)

# Processing loop
for pixel in image_row:
    # Logic: If pixel is bright (>127), set to White
    (255), else Black (0)
    if pixel > threshold_value:
        binary_row.append(255)
    else:
        binary_row.append(0)

print("Thresholded Scanline:", binary_row)
```

Task 2 — The lifecycle of a pixel (dynamic typing).

Python

```
# Micro-Exercise 1.1: The Lifecycle of a Pixel
# Objective: Demonstrate dynamic typing by simulating
pixel normalization.

# 1. Integer: Represents a raw grayscale pixel value (0-255)
pixel_val = 145
print(f"Stage 1 - Raw Input: {pixel_val}, Type: {type(pixel_val)}")
# Expected Output: Type: <class 'int'>

# 2. Float: Normalization (Crucial for Neural Networks)
# Dividing by 255.0 changes the type dynamically to float.
pixel_val = pixel_val / 255.0
print(f"Stage 2 - Normalized: {pixel_val:.4f}, Type: {type(pixel_val)}")
# Expected Output: Type: <class 'float'>

# 3. List: Represents a row of pixels (Scanline)
# Lists are mutable, allowing us to modify pixel values in place.
pixel_row = [145, 200, 34, 255, 0]
print(f"Stage 3 - Pixel Row: {pixel_row}, Type: {type(pixel_row)}")
# Expected Output: Type: <class 'list'>

# 4. Normalize the entire row (element-wise division)
normalized_row = [val / 255.0 for val in pixel_row]
print(f"Stage 4 - Normalized Row: {normalized_row}, Type: {type(normalized_row)}")
# Each element is now a float, but the container is still a list
```

Task 3 — The Detector Hierarchy (a BaseDetector-style class, without OpenCV yet).

Python

```
class FaceDetector:
    def __init__(self, model_path):
        self.model_path = model_path        # Attribute

    def detect(self, image):
        # Logic to detect faces
        pass
```

Python

```
# Instantiation
my_detector = FaceDetector("haarcascade.xml")
```

Task 4 — Data structures for a vision batch (Tuple for immutable constants, Dictionary for class labels, List for a processing queue, Set for unique classes found).

Python

```
from dataclasses import dataclass
from typing import List, Tuple, Set

# 1. TUPLE: Immutable coordinates and colors
BBBOX_COLOR = (0, 255, 0) # RGB Green

# 2. DICTIONARY: Mapping Class IDs to Labels
CLASS_LABELS = {
    0: "cat",
    1: "dog",
    2: "bird"
}

# 3. LIST: A sequence of images to be processed
image_queue = ["frame_001.png", "frame_002.png",
               "frame_003.png"]

@dataclass
class DetectionResult:
    image_name: str
    box: Tuple[int, int, int, int]
    class_id: int

def process_vision_batch(queue: List[str]):
    # Store all unique classes found across the batch
    unique_classes_found: Set[str] = set()
    # Store full metadata for each detection
    batch_results: List[DetectionResult] = []

    for img_name in queue:
        # Simulated raw detection data
        simulated_id = 0 if "001" in img_name else 1
        simulated_box = (10, 10, 100, 100)

        result = DetectionResult(img_name, simulated_box,
                                simulated_id)
```

Python

```
        batch_results.append(result)

        # 4. SET: Adding labels to track unique species
found
        label = CLASS_LABELS[simulated_id]
        unique_classes_found.add(label)

    return batch_results, unique_classes_found

# --- Execution ---
results, summary = process_vision_batch(image_queue)
print(f"--- Processing Summary 2026 ---")
print(f"Total Detections: {len(results)}")
print(f"Unique Categories Detected: {summary}")

first_item = results[0]
print(f"\nFirst Image: {first_item.image_name}")
print(f"Detected: {CLASS_LABELS[first_item.class_id]}")
print(f"Coordinates: {first_item.box}")
```

PART D — Problem-Based Learning

Apply everything above to four small original programs:

1. **Greet 2 of your friends** — different greetings depending on whether it's morning or evening.
2. **The Calculator** — get two numbers as input, then calculate and display their addition, subtraction, multiplication, and division.
3. **You are the Doctor** — write a script that checks (with a rule you define) whether a patient shows signs of COVID-19.
4. **Rock, Paper, Scissors** — write a script that plays a full round of rock-paper-scissors against the computer.

G–J. Result, Discussion, Conclusion, References

Follow the reporting structure in this companion's introduction. References for this lab: OpenCV Tutorial (Great Learning), Programming for Everybody (Coursera), Object-Oriented Programming in Python (DataCamp), and the Python Tutorial on Tutorials Point.

PRACTICAL WORK 2 — Python Basics for Image Processing

CLO 2 · Construct suitable application using machine learning method · **PL04, P4** · DK3, DK4, DK8 / DP1, DP4

Amir's Notebook

With Python installed, Amir's second lab was the first time a picture actually appeared on his screen because *he* told it to. That single moment — `cv2.imshow` popping open a window with his own test photo in it — is usually the point where a diploma project stops feeling abstract.

A. Objectives

1. Load, display, and save images using Python libraries correctly.
2. Apply basic image processing operations such as grayscale conversion and resizing correctly.
3. Apply Python control structures and functions to process image data successfully.
4. Apply Object-Oriented Programming for image processing correctly.

B. Learning Outcome

Use Python to read and manipulate image data, and apply basic image processing techniques as preparation for computer vision applications.

 **Concept Bridge** — *Seeing Like a Machine*, Unit 2.1–2.2 covers the video-loop skeleton and colour-space theory this lab puts into practice.

C. Topic Summary — Fundamentals of Digital Image Processing

A digital image is a two-dimensional grid of **pixels** (picture elements), each storing intensity information representing brightness and colour. Computer vision treats images as numerical matrices, which is what makes mathematical analysis and transformation possible.

In **grayscale images**, each pixel holds a single intensity value from 0 (black) to 255 (white). **Colour images** typically use the **RGB** model, where each pixel carries three intensity values — red, green, and blue.

Image representation in Python. Images are represented as NumPy arrays: rows = height, columns = width, and (for colour) a third dimension for channels. A grayscale image therefore has shape (height, width); a colour image has shape (height, width, 3).

Image processing libraries. This lab introduces **OpenCV (cv2)** for its efficiency and ubiquity in computer vision. OpenCV reads and writes image files, converts between colour spaces, resizes and transforms images, and prepares images for later work such as contour detection.

Grayscale conversion reduces a colour image to a single intensity channel — a standard preprocessing step that lowers computational complexity for edge detection, contour detection, and many ML pipelines.

Image resizing changes an image’s resolution — a critical preprocessing step, since most models require fixed input dimensions. Resizing without preserving the aspect ratio distorts the image and can degrade model accuracy.

Object-Oriented Programming for image processing. The syllabus places heavy emphasis on OOP because it is how real “Detector” or “Tracker” objects are built — objects that keep their own internal state (such as the history of an object’s position) across frames. A *Class* is a blueprint; an *Object* is the specific instance built from it. **Encapsulation** hides implementation details behind naming conventions (`self.name` public, `self._name` protected, `self.__name` private). **Inheritance** lets a `YOLODetector` class derive from a generic `ObjectDetector` class. **Polymorphism** lets different detector classes share one `.detect()` interface. **Magic methods** like `__str__` let an object describe itself when printed.

D. Material / Tools

1. PC installed with Python 3.x and Visual Studio Code
2. Python libraries: NumPy, OpenCV (cv2)
3. CIDOS Online Platform

E. General Instruction / Safety Procedure

See “Before every lab” in the introduction to this companion.

F. Work Instruction / Procedure

PART A — Installing Required Python Libraries

1. Confirm Python 3.x is installed: `python --version`
2. Install the required libraries:

Bash

```
pip install numpy opencv-python
```

3. Verify the installation by opening the interactive interpreter (`python` or `python3`) and running:

Python

```
import numpy as np
```

Python

```
import cv2
print(np.__version__)
print(cv2.__version__)
```

No error messages means the imports succeeded.

4. Exit with `exit()`.

PART B — Task 1: Loading and Displaying an Image

1. Download a sample image, e.g. <https://www.gstatic.com/webp/gallery/3.jpg>, and place it in your project folder.
2. Write the following code:

Python

```
# Task 1: Loading and Displaying an Image
# Objective: To read and display an image using python
import cv2
import os

# Use a path relative to this script so it works on any
machine/lab PC
image_path =
os.path.join(os.path.dirname(os.path.abspath(__file__)),
"3.jpg")
# You may also hardcode a full path, e.g. r'C:\Users\
<you>\Projects\Lab2\3.jpg'

if not os.path.exists(image_path):
    print(f"Error: Image file not found at"
    '{image_path}')
else:
    img = cv2.imread(image_path)
    if img is None:
        print("Error: The image cannot be read. Check"
        file permissions.")
    else:
        cv2.imshow("Image Display", img)
        cv2.waitKey(0)
        cv2.destroyAllWindows()
```

 **Fixed in this Companion** — The original code hardcoded an absolute Windows path (`r'C:\Users\zoolf\...`) that only works on one specific laptop. This version builds the path relative to the script's own folder with `os.path.dirname(os.path.abspath(__file__))`, so the exact same script works on

any lab PC, your own laptop, or your groupmate's machine — you only need to keep 3.jpg in the same folder as the script. You can still hardcode a full path if you prefer; just remember to replace it with your own.

PART B — Task 2: Grayscale Conversion

Python

```
# Task 2: Grayscale Conversion
# Objective: To Convert a color image into grayscale
import cv2
import os

image_path =
os.path.join(os.path.dirname(os.path.abspath(__file__)),
"3.jpg")

if not os.path.exists(image_path):
    print(f"Error: Image file not found at
'{image_path}'")
else:
    img = cv2.imread(image_path)
    if img is None:
        print("Error: The image cannot be read. Check
file permissions.")
    else:
        gray_image = cv2.cvtColor(img,
cv2.COLOR_BGR2GRAY)
        cv2.imshow("Original Image", img)
        cv2.imshow("Grayscale Image", gray_image)
        cv2.waitKey(0)
        cv2.destroyAllWindows()
```

PART B — Task 3: Image Resizing (preserving aspect ratio)

Python

```
# Task 3: Resize Image
# Objective: To Resize an image while maintaining aspect
ratio
import cv2
import os

image_path =
os.path.join(os.path.dirname(os.path.abspath(__file__)),
"3.jpg")
```

Python

```
if not os.path.exists(image_path):
    print(f"Error: Image file not found at '{image_path}'")
else:
    img = cv2.imread(image_path)
    if img is None:
        print("Error: The image cannot be read. Check file permissions.")
    else:
        height, width = img.shape[:2]
        new_width = 400
        scale_ratio = new_width / width
        new_height = int(height * scale_ratio)

        resized_img = cv2.resize(img, (new_width, new_height))
        cv2.imshow("Resized Image", resized_img)
        cv2.waitKey(0)
        cv2.destroyAllWindows()
```

PART B — Task 4: Saving Processed Images

Add this before your `waitKey(0)` / `destroyAllWindows()` calls in Tasks 2 and 3:

Python

```
cv2.imwrite("output_gray.jpg", gray_image)
cv2.imwrite("output_resized.jpg", resized_img)
```

PART C — Task 1: Control Structures and Functions

This demonstrates functions, conditionals, and loops processing image data programmatically — resizing the same image to three different widths in a batch.

Python

```
import cv2
import os

def process_image(image, new_width):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    height, width = image.shape[:2]
    scale = new_width / width
    new_height = int(height * scale)
```

Python

```
        resized = cv2.resize(gray, (new_width, new_height))
        return resized

image_path =
os.path.join(os.path.dirname(os.path.abspath(__file__)),
"3.jpg")
image = cv2.imread(image_path)

if image is None:
    print("Error: Image not found")
else:
    sizes = [300, 400, 500]
    for size in sizes:
        output = process_image(image, size)
        filename = f"output_gray_{size}.jpg"
        cv2.imwrite(filename, output)
        print(f"Saved {filename}")
        cv2.imshow(f"Grayscale Resized {size}", output)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

 **Fixed in this Companion** — In the original version, `cv2.waitKey(0)` and `cv2.destroyAllWindows()` were indented *inside* the `for` loop, so the script paused and waited for a keypress after every single resize before moving to the next one. This version calls them once, after the loop finishes, so all three windows open together and the script waits for one keypress at the end.

PART C — Task 2: OOP for Image Processing

This wraps the loading, grayscale conversion, resizing, and saving logic into a reusable `ImageProcessor` class, plus a `batch_process()` helper — directly satisfying the syllabus requirement to apply OOP to image processing.

Python

```
import cv2
import os

class ImageProcessor:
    """Encapsulates common image processing operations
    using OOP."""

    def __init__(self, image_path):
        self.image_path = image_path
        self.image = None
```

Python

```
def load(self):
    print(f"[INFO] Loading: {self.image_path}")
    self.image = cv2.imread(self.image_path)
    if self.image is None:
        raise FileNotFoundError(f"[ERROR] Cannot read
image: {self.image_path}")
    print(f"[OK] Loaded image shape:
{self.image.shape}")
    return self.image

def to_gray(self):
    if self.image is None:
        raise ValueError("[ERROR] Image not loaded.
Call load() first.")
    gray = cv2.cvtColor(self.image,
cv2.COLOR_BGR2GRAY)
    print("[OK] Converted to grayscale")
    return gray

def resize_keep_aspect(self, new_width):
    if self.image is None:
        raise ValueError("[ERROR] Image not loaded.
Call load() first.")
    h, w = self.image.shape[:2]
    scale = new_width / w
    new_height = int(h * scale)
    resized = cv2.resize(self.image, (new_width,
new_height))
    print(f"[OK] Resized to: {resized.shape}")
    return resized

def save(self, img, filename):
    ok = cv2.imwrite(filename, img)
    if not ok:
        raise IOError(f"[ERROR] Failed to save:
{filename}")
    print(f"[SAVED] {filename}")

def batch_process(input_files, out_dir="outputs",
widths=(300, 400, 500), show=False):
    os.makedirs(out_dir, exist_ok=True)
    print(f"[INFO] Output folder:
```

Python

```
{os.path.abspath(out_dir)}")

    for path in input_files:
        proc = ImageProcessor(path)
        proc.load()
        gray = proc.to_gray()

        base = os.path.splitext(os.path.basename(path))
[0]
        gray_path = os.path.join(out_dir,
f"{base}_gray.jpg")
        proc.save(gray, gray_path)

        for w in widths:
            resized = proc.resize_keep_aspect(w)
            resized_path = os.path.join(out_dir,
f"{base}_resized_{w}.jpg")
            proc.save(resized, resized_path)

        if show:
            cv2.imshow("Original", proc.image)
            cv2.imshow("Gray", gray)
            cv2.waitKey(0)
            cv2.destroyAllWindows()

    print("[DONE] Batch processing complete.")

if __name__ == "__main__":
    script_dir =
os.path.dirname(os.path.abspath(__file__))
    batch_process(
        [os.path.join(script_dir, "3.jpg")],
        out_dir="outputs",
        widths=(300, 400, 500),
        show=False    # set True if you want cv2 windows
    )
```

Where to find the results: the script prints the full output folder path, e.g. [INFO] Output folder: C:\...\outputs. Inside it you'll find 3_gray.jpg, 3_resized_300.jpg, 3_resized_400.jpg, and 3_resized_500.jpg.

PART D — Problem-Based Learning

1. **Debugging Challenge** — your lecturer will supply a broken script; find and fix every bug.

2. **The Dice Game** — simulate a two-dice roll using Python's `random` module and print the outcome (and, if you like, a win condition).
3. **Starting Your Engine: Build the Module and Test the Fleet** — refactor your image-processing functions from Part C into their own importable module (e.g. `vision_utils.py`), then write a short “fleet test” script that imports it and runs it against three different images.

G–J. Result, Discussion, Conclusion, References

Follow the reporting structure in this companion's introduction. References: OpenCV Tutorial (Great Learning), Programming for Everybody (Coursera).

PRACTICAL WORK 3 — OpenCV Image and Video Processing in Vision Application

CLO 2 · Construct suitable application using machine learning method · **PLO4, P4** · DK3, DK4, DK8 / DP1, DP4

Amir's Notebook

PW3 is where Amir's guardhouse project stopped being separate little scripts and started becoming *one system*: a camera that could isolate a colour, follow it, read text off a page, and scan a QR pass — all in the same afternoon of work. It is also, by far, the longest lab in the semester. Budget real time for it.

A. Objectives

1. Install and set up OpenCV using Python code to process still image and video sources.
2. Apply image pixel representation using NumPy, and BGR–GRAY / BGR–HSV conversion correctly.
3. Apply masking, image blurring, Canny edge detection, and contour detection successfully.
4. Apply image processing to create an application for object tracking, Optical Character Recognition (OCR), and QR code/Barcode detection correctly.

B. Learning Outcome

Use Python to read and manipulate image data; apply basic image processing techniques; and apply image processing to build object tracking, OCR, and QR/barcode applications.

 **Concept Bridge** — this lab is the hands-on twin of *Seeing Like a Machine* Unit 2 in full: colour spaces (2.2), masking and blurring (2.3.1–2.3.2), Canny edge detection (2.3.3), contours (2.4), and Lab Blueprints A–C (2.5).

C. Topic Summary — OpenCV and Python Applications

OpenCV (Open Source Computer Vision Library) is the premier real-time computer vision toolkit, from simple filtering to complex geometric transformations. All code in this lab targets OpenCV 4. OpenCV's Python distribution ships as `opencv-python` (core modules) or `opencv-contrib-python` (adds extra/contrib modules); headless variants exist for servers or containers without GUI support. The introductory workflow is always the same loop: read an image, display it, write it to disk.

Coordinate system and colour spaces. A digital image is numbers arranged in rows and columns; grayscale intensity typically ranges 0 (black) to 255 (white) in 8-bit images. OpenCV often *talks about* “RGB,” but the default channel order it actually loads is **BGR** (blue, green, red) unless you explicitly convert. As a NumPy array, a loaded image has shape (`height, width, channels`) and dtype `uint8` — and contour data returned by OpenCV is *also* NumPy arrays of coordinate points, reinforcing the idea that “images and geometry are both just arrays.”

BGR→GRAY is one of the most common conversions, always through `cvtColor`. **BGR→HSV** is commonly paired with `inRange()` colour thresholding to isolate objects by colour, since HSV separates *colour identity* (hue) from *light intensity* (value) far better than BGR does — RGB is the standard for editing/display (Matplotlib, PIL, monitors); BGR is OpenCV's default and older Windows BMP's default.

Masking, blurring, Canny. A mask is a binary image where white (255) marks the region to keep and black (0) marks what to discard; `cv2.bitwise_and()` applies it. `inRange()` is the standard thresholding method for detecting objects by pixel range (commonly in HSV). Blurring covers several families — box/mean blur, Gaussian blur, median blur (best against salt-and-pepper noise), and bilateral filtering (smooths while preserving edges). Canny edge detection is a multi-stage algorithm — `cv2.Canny(image, threshold1, threshold2)` — widely regarded as the standard for edge detection due to its detection accuracy, localisation, and single-response behaviour.

Contours are boundary curves found on binary images; you typically want your object shown in white on a black background (or as edges from Canny) for best results. `findContours()` returns a list of contours; `drawContours()` draws them. Contour **approximation** (`approxPolyDP`, based on the Douglas–Peucker algorithm) simplifies a contour into a similar polygon with fewer vertices, controlled by a precision parameter (epsilon) — this is how you go from “a blob of points” to “this is a rectangle.”

Building applications from these blocks. Chaining these modular pieces — object tracking, OCR, and barcode/QR recognition — using OpenCV, YOLO, or frameworks like TensorFlow/Keras, is what converts a raw video feed into actionable data for industrial, inventory, or surveillance use cases, often running on an edge device or a phone.

D. Material / Tools

1. PC installed with Python 3.x and Visual Studio Code
2. Python libraries: NumPy, OpenCV (cv2)
3. CIDOS Online Platform

E. General Instruction / Safety Procedure

See “Before every lab” in the introduction to this companion.

F. Work Instruction / Procedure

PART A — Activate a Virtual Environment in VS Code

1. Create and activate a virtual environment (`venv`/`virtualenv`/`conda`) so this lab’s dependencies stay isolated from other projects.
2. **Recommended: use VS Code’s Command Palette.** Press **Ctrl+Shift+P** (Windows/Linux) or **Cmd+Shift+P** (macOS).
3. Type and select **Python: Create Environment...**
4. Choose **Venv** (or Conda, if you use it).
5. Choose your base Python 3.x interpreter from the discovered list.
6. VS Code creates a `.venv` folder in your workspace and selects it automatically as your project’s interpreter — you’ll see a confirming notification and the interpreter shown in the status bar.
7. **Activate the environment.** Once selected, opening a *new* terminal in VS Code activates it automatically — you’ll see (`.venv`) at the start of the prompt. If you need to activate it manually in an existing terminal:
 - Windows (Command Prompt/PowerShell): `.\venv\Scripts\activate`
 - macOS/Linux (Bash/Zsh): `source venv/bin/activate`
8. **Install dependencies** into this new environment:

Bash

```
pip install numpy opencv-python
```

PART B — Task 1: Construct Code to Process a Still Image

This generates a synthetic demo image entirely in code (no download needed), saves it, reloads it, and displays it — proving the whole `imread/imshow/waitKey/imwrite` loop works before you touch a real photo.

Python

```
# Task 1: Construct code to process a still image
# Objective: To generate a demo image, save it, reload
it, and display it using python
import cv2
import numpy as np

def make_demo_scene(width=640, height=400):
    """
    Create a synthetic BGR image so you can practice
    without downloading files.
    """
    img = np.zeros((height, width, 3), dtype=np.uint8)
    img[:] = (30, 30, 30) # dark gray background (BGR)

    # Draw colored shapes (BGR order!)
    cv2.rectangle(img, (30, 40), (220, 200), (0, 0, 255),
-1) # red rectangle
    cv2.circle(img, (340, 120), 70, (0, 255, 0), -1)
# green circle
    pts = np.array([[470, 220], [600, 220], [535, 320]],
np.int32)
    cv2.fillPoly(img, [pts], (255, 0, 0))
# blue triangle

    cv2.putText(img, "OpenCV demo", (30, 330),
cv2.FONT_HERSHEY_SIMPLEX, 1.0, (255, 255,
255), 2, cv2.LINE_AA)
    return img

img = make_demo_scene()
cv2.imwrite("demo.png", img) # save (BGR PNG)
loaded = cv2.imread("demo.png") # read back

if loaded is None:
    raise FileNotFoundError("Failed to read demo.png
(check working directory).")

cv2.imshow("Loaded image", loaded)
cv2.waitKey(0) # keep window open
until a key press
cv2.destroyAllWindows()
```

Add the generated image to your report. Then, using GitHub Copilot in VS Code, refactor `make_demo_scene()` into a class with methods to add each shape (e.g. `add_rectangle()`, `add_circle()`, `add_triangle()`) — add the resulting code and its output to your report too.

PART B — Task 2: Loading an Image and Its Pixel Representation with NumPy

1. Download `marsrover.png` from the course GitHub repository:
<https://github.com/zoolfahmy/DEC50302-MACHINE-LEARNING-IN-COMPUTER-VISION/blob/myBranch/marsrover.png>
2. Place it in the same project folder.
3. Write this code:

Python

```
# Task 2: Loading and Displaying an Image
# Objective: To read and display an image using numpy
import cv2
import os

image_path =
os.path.join(os.path.dirname(os.path.abspath(__file__)),
"marsrover.png")

# Read or load image from its path
image = cv2.imread(image_path)

if image is None:
    raise FileNotFoundError(f"Could not read image at
'{image_path}'. Check the path and filename.")

# image is a NumPy array
print("Dimension of the image: ", image.ndim)
print("Image height: ", format(image.shape[0]))
print("Image width: ", format(image.shape[1]))
print("Image channels: ", format(image.shape[2]))
print("Size of the image array: ", image.size)

# Display the image and wait until a key is pressed
cv2.imshow("My Image", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

 **Fixed in this Companion** — The original code called `image.ndim` and `image.shape` immediately after `cv2.imread()` with no check that loading actually succeeded. If the path was even slightly wrong, this crashed with a confusing `AttributeError: 'NoneType' object has no attribute 'ndim'` instead of telling you the real problem. This version raises a clear `FileNotFoundError` naming the exact path it tried, so you can fix the real issue in seconds.

Add the image and the printed dimensions to your report. Then, using GitHub Copilot, change the code to show individual channels: `b, g, r = cv2.split(image)`, then `cv2.imshow()` each channel separately to see the BGR channels in isolation. Add that code and output to your report.

PART B — Task 3: Loading and Displaying a Video (Grayscale Conversion)

Python

```
# Task 3: Loading and Displaying a Video
# Objective: To Convert a color of video into grayscale
import cv2

cap = cv2.VideoCapture(0) # 0 = default camera; for a
file: cv2.VideoCapture("video.mp4")

if not cap.isOpened():
    raise RuntimeError("Could not open camera/video
source.")

while True:
    ret, frame = cap.read()
    if not ret:
        break
    cv2.imshow("Original video", frame)
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    cv2.imshow("Grayscale video", gray)

    # press 'q' to quit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Run the file, press **q** to quit, and add a screenshot of the output to your report. Then, using GitHub Copilot, extend the code to draw a bounding box tracking your face (hint: a Haar cascade classifier, the same technique used later in *Seeing Like a Machine* Unit 4.2). Add that code and its output to your report.

PART B — Task 4: BGR to GRAY Conversion

1. Download `nature.jpg`: <https://github.com/zoolfahmy/DEC50302-MACHINE-LEARNING-IN-COMPUTER-VISION/blob/myBranch/nature.jpg>
2. Place it in your project folder.

Python

```
# Task 4: BGR to GRAY Conversion
# Objective: To Conversion image from BGR to Gray
import cv2
import os
```

Python

```
image_path =
os.path.join(os.path.dirname(os.path.abspath(__file__)),
"nature.jpg")
img = cv2.imread(image_path)
if img is None:
    raise FileNotFoundError("Provide nature.jpg (or check
working directory).")

cv2.imshow("Original image", img)

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

cv2.imwrite("demo_gray.png", gray)
print("Gray shape:", gray.shape)    # (H, W)
cv2.imshow("Grayscale image", gray)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Add the output image to your report. Then, using GitHub Copilot, try the different conversion codes `cv2.COLOR_BGR2GRAY`, `cv2.COLOR_RGB2GRAY`, `cv2.COLOR_BGR2LAB`, and `cv2.COLOR_BGR2HSV` — add the code and outputs for each to your report.

PART B — Task 5: BGR to HSV Conversion (Grayscale Histogram)

1. Download `park.jpg`: <https://github.com/zoolfahmy/DEC50302-MACHINE-LEARNING-IN-COMPUTER-VISION/blob/myBranch/park.jpg>
2. Place it in your project folder.
3. Install Matplotlib in your environment: `pip install matplotlib`
4. Write this code:

Python

```
# Task 5: BGR to HSV Conversion
# Objective: Grayscale Histogram
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

image_path =
os.path.join(os.path.dirname(os.path.abspath(__file__)),
"park.jpg")
img = cv2.imread(image_path)
if img is None:
    raise FileNotFoundError("Provide park.jpg (or check
```

Python

```
working directory).")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
cv2.imshow("Grayscale image", gray)

hist = cv2.calcHist([gray], [0], None, [256], [0,
256]).flatten()

plt.figure()
plt.title("Grayscale intensity histogram")
plt.xlabel("Pixel intensity (0-255)")
plt.ylabel("Count")
plt.plot(hist)
plt.show()
```

Add the image and histogram to your report.

PART C — Task 1: Masking Technique (Isolate a Colour)

1. Download green_tree.jpg: <https://github.com/zoolfahmy/DEC50302-MACHINE-LEARNING-IN-COMPUTER-VISION/blob/myBranch/green%20tree.jpg>
2. Place it in your project folder.

Python

```
# Part C Task 1: Masking Technique
# Objective: isolate the green object in the image
import cv2
import numpy as np
import os

image_path =
os.path.join(os.path.dirname(os.path.abspath(__file__)),
"green_tree.jpg")
img = cv2.imread(image_path)
if img is None:
    raise FileNotFoundError("Provide 'green_tree.jpg' (or
check working directory).")

cv2.imshow("Original image", img)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Hue ranges depend on lighting/camera. These are starter
values for "green".
lower_green = np.array([35, 80, 80], dtype=np.uint8)
upper_green = np.array([85, 255, 255], dtype=np.uint8)
```


Python

```
mask = cv2.inRange(hsv, lower_green, upper_green)

# Apply the mask: keep only pixels where mask != 0
segmented = cv2.bitwise_and(img, img, mask=mask)

cv2.imwrite("mask.png", mask)
cv2.imshow("Mask", mask)
cv2.imwrite("segmented_green.png", segmented)
cv2.imshow("Segmented green areas", segmented)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Add the mask and segmented image to your report. Then, using GitHub Copilot: change `lower_green/upper_green` to catch other colours (red, blue, yellow), and add interactive trackbars (`cv2.createTrackbar`) so you can adjust the HSV bounds live and watch the mask update. Add the code and output to your report.

PART C — Task 2: Image Blurring Method (Compare Blur Types)

Uses `park.jpg` again.

Python

```
# Part C Task 2: Image Blurring Method
# Objective: To Compare blur type
import cv2
import os

image_path =
os.path.join(os.path.dirname(os.path.abspath(__file__)),
"park.jpg")

if not os.path.exists(image_path):
    print(f"Error: Image file not found at '{image_path}'")
else:
    img = cv2.imread(image_path)
    if img is None:
        print("Error: The image cannot be read. Check file permissions.")
    else:
        box = cv2.blur(img, (7, 7))
        gauss = cv2.GaussianBlur(img, (7, 7), 0)
        median = cv2.medianBlur(img, 7)
```

Python

```
    bilateral = cv2.bilateralFilter(img, d=9,
sigmaColor=75, sigmaSpace=75)

    cv2.imwrite("blur_box.png", box)
    cv2.imshow("Box blur", box)
    cv2.imwrite("blur_gaussian.png", gauss)
    cv2.imshow("Gaussian blur", gauss)
    cv2.imwrite("blur_median.png", median)
    cv2.imshow("Median blur", median)
    cv2.imwrite("blur_bilateral.png", bilateral)
    cv2.imshow("Bilateral blur", bilateral)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

Add the output images to your report. Then, using GitHub Copilot: implement your own box blur using convolution (`cv2.filter2D` with a normalised kernel), and write a function that returns a dictionary of blurred versions for unit testing. Add the code and output to your report.

PART C — Task 3: Canny Edge Detection (with a Tuning Graph)

This task produces (1) an edge image and (2) an “edge pixel count vs threshold” graph, so you can see how Canny’s sensitivity changes.

1. Download `soccer-in-green.jpg`: <https://github.com/zoolfahmy/DEC50302-MACHINE-LEARNING-IN-COMPUTER-VISION/blob/myBranch/soccer-in-green.jpg>
2. Place it in your project folder.

Python

```
# Part C Task 3: Canny Edge Detection
# Objective: To run Canny and produce a tuning graph
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

image_path =
os.path.join(os.path.dirname(os.path.abspath(__file__)),
"soccer-in-green.jpg")
img = cv2.imread(image_path)
if img is None:
    raise FileNotFoundError("Provide 'soccer-in-
green.jpg' (or check working directory).")

cv2.imshow("Original image", img)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Python

```
def edge_count(t1, t2):
    edges = cv2.Canny(gray, t1, t2)
    return int((edges > 0).sum()), edges

# Save and show one example edge map
count, edges = edge_count(50, 150)
cv2.imwrite("edges_50_150.png", edges)
cv2.imshow("Canny edges (t1=50, t2=150)", edges)
print(f"Edge pixels detected with t1=50, t2=150: {count}")

# Sweep thresholds and plot edge counts
t1_values = list(range(10, 200, 10))
counts = []
for t1 in t1_values:
    t2 = min(255, 3 * t1) # common heuristic: high
    # threshold ~ 2-3x low threshold
    c, _ = edge_count(t1, t2)
    counts.append(c)

plt.figure()
plt.title("Canny sensitivity curve (soccer-in-green.jpg)")
plt.xlabel("Lower threshold (t1)")
plt.ylabel("Edge pixels detected")
plt.plot(t1_values, counts)
plt.show()
```

 **Fixed in this Companion** — The original code loaded the image and called `cv2.cvtColor(img, ...)` without ever checking whether `img` came back as `None`. If the filename or path was even slightly off, the script crashed with a cryptic error inside OpenCV's C++ layer instead of a clear message. This version checks immediately after loading and raises a plain-English `FileNotFoundError` instead.

Add the edge image and the sensitivity graph to your report. Then, using GitHub Copilot: colourise the edges by mapping edge strength to a heatmap (hot = strong, cool = weak). Add the code and output to your report.

PART C — Task 4(i): Contour Drawing

This creates a clean shapes-only image (white shapes on black) for predictable contours — you typically want the object in white on a black background for the best results.

Python

```
# Part C Task 4(i): Contour Drawing
# Objective: To Draw shape contour
import cv2
import numpy as np

canvas = np.zeros((400, 640, 3), dtype=np.uint8)
cv2.rectangle(canvas, (50, 50), (220, 220), (255, 255, 255), -1)
cv2.circle(canvas, (350, 140), 80, (255, 255, 255), -1)
pts = np.array([[480, 250], [600, 250], [540, 350]],
               np.int32)
cv2.fillPoly(canvas, [pts], (255, 255, 255))

gray = cv2.cvtColor(canvas, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 127, 255,
                          cv2.THRESH_BINARY)

contours, hierarchy = cv2.findContours(binary,
                                       cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

out = canvas.copy()
cv2.drawContours(out, contours, -1, (0, 255, 0), 3)

cv2.imwrite("shapes_binary.png", binary)
cv2.imshow("Binary image", binary)
cv2.imwrite("shapes_contours.png", out)
cv2.imshow("Contours drawn", out)

print("Contours found:", len(contours))
for i, c in enumerate(contours):
    print(f"Contour {i}: {len(c)} points,
          area={cv2.contourArea(c)}")
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Add the generated `shapes_binary.png` and `shapes_contours.png` to your report — you'll find them in your VS Code Explorer panel once the script runs.

PART C — Task 4(ii): Contour Detection and Approximation

Requires Task 4(i) to have run first, since it uses `shapes_contours.png`.

Python

```
# Part C Task 4(ii): Contour Detection and approximation
# Objective: To approximate shapes and label them
```

Python

```
import cv2
import numpy as np

img = cv2.imread("shapes_contours.png")
if img is None:
    raise FileNotFoundError("Run the previous contour
script first to create shapes_contours.png.")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 10, 255,
cv2.THRESH_BINARY) # low threshold since background is
black-ish

contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

out = img.copy()

for c in contours:
    peri = cv2.arcLength(c, True)
    epsilon = 0.02 * peri # 2% of perimeter is a common
starting point
    approx = cv2.approxPolyDP(c, epsilon, True)

    x, y, w, h = cv2.boundingRect(approx)
    vertices = len(approx)

    if vertices == 3:
        label = "triangle"
    elif vertices == 4:
        label = "quadrilateral"
    else:
        label = f"{vertices}-gon/circle-ish"

    cv2.drawContours(out, [approx], -1, (0, 0, 255), 2)
    cv2.putText(out, label, (x, y - 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255,
0), 2)

cv2.imwrite("shapes_approximated.png", out)
cv2.imshow("Approximated contours", out)
cv2.waitKey(0)
```

Add the output to your report. Then, using GitHub Copilot: distinguish squares from rectangles by aspect ratio ($ar = w / \text{float}(h)$, thresholded near 1.0), and detect circles with

`cv2.minEnclosingCircle()`, comparing the contour's area against the enclosing circle's area. Add the code and output to your report.

PART D — Task 1: Object Tracking with HSV + Contours

Tracks the largest green object live in your webcam feed — the same colour-tracking pipeline used in Amir's guardhouse Lab Blueprint A.

Python

```
# Part D Task 1: Object Tracking with HSV + Contours
# Objective: To Track the largest "green" object in
webcam video
import cv2
import numpy as np

cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise RuntimeError("Could not open camera.")

lower_green = np.array([35, 80, 80], dtype=np.uint8)
upper_green = np.array([85, 255, 255], dtype=np.uint8)

kernel = np.ones((5, 5), np.uint8)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, lower_green, upper_green)

    # Clean up mask: opening removes small noise; closing
fills small holes
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel,
iterations=1)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE,
kernel, iterations=1)

    contours, _ = cv2.findContours(mask,
cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    if contours:
        c = max(contours, key=cv2.contourArea)
        area = cv2.contourArea(c)

        # Ignore tiny detections
```

Python

```
    if area > 500:
        x, y, w, h = cv2.boundingRect(c)
        cv2.rectangle(frame, (x, y), (x + w, y + h),
(0, 255, 0), 2)

        M = cv2.moments(c)
        if M["m00"] != 0:
            cx = int(M["m10"] / M["m00"])
            cy = int(M["m01"] / M["m00"])
            cv2.circle(frame, (cx, cy), 5, (0, 0,
255), -1)

            cv2.putText(frame, f"area={int(area)}",
(x, y - 10),
                                cv2.FONT_HERSHEY_SIMPLEX,
0.6, (0, 255, 0), 2)

        cv2.imshow("Tracking (press q to quit)", frame)
        cv2.imshow("Mask", mask)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

cap.release()
cv2.destroyAllWindows()
```

Snap the tracking output and add it to your report. Then, using GitHub Copilot: track different colours (adjust `lower_*/upper_*` for red, blue, yellow — remember red needs two HSV ranges because it wraps around hue 0/179), track *by shape* (approximate contours with `approxPolyDP` and only track shapes with 3/4/5 vertices), and combine colour and shape (e.g. detect green circles, red triangles). Add the code and output to your report.

PART D — Task 2(i): OCR with OpenCV Preprocessing and a TensorFlow Digit Model

This uses OpenCV to manipulate images and TensorFlow to build the recognition model — the classic OCR pipeline: preprocess (OpenCV) — recognise (TensorFlow).

1. Install TensorFlow: `pip install tensorflow`
2. Write this code:

Python

```
# Part D Task 2: OCR with OpenCV preprocessing and a
TensorFlow digit model
# Objective: To train a simple MNIST digit classifier
import tensorflow as tf
import numpy as np
```

Python

```
import matplotlib.pyplot as plt

# Load MNIST as NumPy arrays
(train_x, train_y), (test_x, test_y) =
tf.keras.datasets.mnist.load_data()

# Normalize to [0,1] and add channel dimension: (N, 28,
28, 1)
train_x = (train_x.astype("float32") / 255.0)[..., None]
test_x = (test_x.astype("float32") / 255.0)[..., None]

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28, 1)),
    tf.keras.layers.Conv2D(16, 3, activation="relu"),
    tf.keras.layers.MaxPool2D(),
    tf.keras.layers.Conv2D(32, 3, activation="relu"),
    tf.keras.layers.MaxPool2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax"),
])

model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

history = model.fit(train_x, train_y,
                    validation_split=0.1,
                    epochs=5,
                    batch_size=128,
                    verbose=2)

# Graph: training curves
plt.figure()
plt.title("MNIST training accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.plot(history.history["accuracy"], label="train")
plt.plot(history.history["val_accuracy"], label="val")
plt.legend()
plt.show()

# Save the model (recommended high-level Keras format)
model.save("digit_model.keras")
```


Snap the training process and the graph, and add both to your report. The `digit_model.keras` file is visible in the VS Code Explorer once training finishes.

PART D — Task 2(ii): OCR Inference on a Single Digit Image

Uses a photo named `digit4.png` containing a single handwritten digit — write a digit, photograph it, and crop it.

Python

```
# Part D Task 2(ii): OCR inference on a single digit image using OpenCV preprocessing
# Objective: To predict a digit using the trained MNIST digit_model.keras
import cv2
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import os

if not os.path.exists("digit_model.keras"):
    raise FileNotFoundError("Run Task 2(i) first to produce digit_model.keras.")

model = tf.keras.models.load_model("digit_model.keras")

img = cv2.imread("digit4.png")
if img is None:
    raise FileNotFoundError("Provide a digit4.png image with one digit to classify.")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# Normalize lighting and binarize
gray = cv2.GaussianBlur(gray, (5, 5), 0)
_, th = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Optional cleanup: remove small noise, close small gaps
kernel = np.ones((3, 3), np.uint8)
th = cv2.morphologyEx(th, cv2.MORPH_OPEN, kernel, iterations=1)

# Resize to MNIST size and normalize
roi = cv2.resize(th, (28, 28), interpolation=cv2.INTER_AREA)
roi = roi.astype("float32") / 255.0
roi = roi[None, ..., None] # (1, 28, 28, 1)
```

Python

```
# See what the model actually sees
plt.figure(figsize=(4, 4))
plt.imshow(roi[0, :, :, 0], cmap='gray')
plt.title("What the model sees")
plt.show()

probs = model.predict(roi, verbose=0)[0]
pred = int(np.argmax(probs))
print("Prediction:", pred)
print("Probabilities:", probs.round(3))
```

 **Fixed in this Companion** — Added a check that `digit_model.keras` exists before trying to load it, with a message pointing you back to Task 2(i) — otherwise the original error (a raw file-not-found exception from deep inside TensorFlow) makes it look like something is broken, when really the model just hasn't been trained yet.

Snap the predicted output and add it to your report.

PART D — Task 3: QR Detection on a Webcam Stream

Uses the `QRCodeDetector` API, which can both detect and decode via `detectAndDecode()`, returning an empty string if decoding fails.

Python

```
# Part D Task 3: QR detection on webcam stream
# Objective: To detect QR using webcam
import cv2

detector = cv2.QRCodeDetector()
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    raise RuntimeError("Could not open camera.")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    text, points, _ = detector.detectAndDecode(frame)

    if points is not None and len(points) > 0:
        pts = points.astype(int).reshape(-1, 2)
        for i in range(len(pts)):
```

Python

```
        cv2.line(frame, tuple(pts[i]), tuple(pts[(i +
1) % len(pts)]), (0, 255, 0), 2)

    if text:
        cv2.putText(frame, text, (20, 40),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0,
255, 0), 2)

    cv2.imshow("QR scanner (press q to quit)", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Snap the scanned result and add it to your report. Then, using GitHub Copilot: add a beep or sound on successful decode (`winsound.Beep` on Windows, or play a short audio file), and add frame-to-frame tracking (once a QR is detected, track its bounding box with a CSRT/KCF tracker to reduce repeated decoding and improve FPS). Add the code and output to your report.

PART D — Task 4: Barcode Detection + Decode with Type

Uses `BarcodeDetector`, `detect`, `decode`, or a combined call — and OpenCV's documented compatibility constraints and supported barcode families.

1. Download `BarcodeC128.png` and `Barcode_ean13.png` from the course GitHub repository (same repo as above), and place them in your project folder.
2. Install `pyzbar`: `pip install pyzbar`
3. Write this code:

Python

```
# Part D Task 4: Barcode Detection
# Objective: To detect barcode and decode
import os
import cv2
import numpy as np
from pyzbar.pyzbar import decode

print("cwd:", os.getcwd())
print("cv2:", cv2.__version__, "barcode module?",
hasattr(cv2, "barcode"))

bardet = cv2.barcode.BarcodeDetector()

img_path = "BarcodeC128.png" # change this to test the
```

Python

```
other file
img = cv2.imread(img_path)
if img is None:
    raise FileNotFoundError(f"cannot open image
{img_path}")

def decode_with_fallback(img):
    # try OpenCV first
    ok, info, corners =
bardet.detectAndDecodeWithType(img)
    if ok and any(s for s in info):
        if corners is not None:
            for quad in corners:
                pts = quad.astype(int).reshape(-1, 2)
                cv2.polylines(img, [pts.reshape(-1, 1,
2)], True, (0, 255, 0), 2)
            return True, tuple(info), tuple([]), img

    # fallback to pyzbar
    res = decode(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY))
    if not res:
        return False, (), (), img

    decoded = [r.data.decode("utf-8") for r in res]
    types = [r.type for r in res]
    for r in res:
        pts = np.array([(p.x, p.y) for p in r.polygon],
np.int32).reshape(-1, 1, 2)
        cv2.polylines(img, [pts], True, (0, 255, 0), 2)
        cv2.putText(img, decoded[0], (r.rect.left, max(0,
r.rect.top - 10)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0,
255, 0), 2)
        return True, tuple(decoded), tuple(types), img

    # try original + four rotations; save first successful
    annotated image
    for angle, flag in [(0, None), (90,
cv2.ROTATE_90_CLOCKWISE),
                        (180, cv2.ROTATE_180), (270,
cv2.ROTATE_90_COUNTERCLOCKWISE)]:
        test = img if flag is None else cv2.rotate(img, flag)
```

Python

```
ok, info, types, annotated =  
decode_with_fallback(test.copy())  
print(f"{angle:3d} decode -> {ok} {info} {types}")  
if ok:  
    out_name = f"decoded_{angle}.png"  
    cv2.imwrite(out_name, annotated)  
    print("Saved:", out_name)  
    break
```

Change `img_path` to test both barcode images. Snap the decoded, annotated output and add it to your report. Then, using GitHub Copilot: run both OpenCV and pyzbar independently on the same image and compare success rates, decode time, and accuracy; and build a small test harness that runs across multiple barcode images and logs success/failure/confidence metrics to a CSV. Add the code and output to your report.

G–J. Result, Discussion, Conclusion, References

Follow the reporting structure in this companion's introduction. **Where to find the results:** your project folder and `outputs/working` directory should contain `green_tree.jpg`, `marsrover.png`, `nature.jpg`, `park.jpg`, `soccer-in-green.jpg`, `BarcodeC128.png`, and `Barcode_ean13.png` alongside every generated output file. References: OpenCV Tutorial (Great Learning), Programming for Everybody (Coursera), and Ansari (2023) — *Building Computer Vision Applications Using Artificial Neural Networks* (2nd ed.), Apress.

PRACTICAL WORK 4 — Model Training Using Teachable Machine & TensorFlow

CLO 2 · Construct suitable application using machine learning method · **PLO4, P4** · DK3, DK4, DK8 / DP1, DP4

Amir's Notebook

For three labs, Amir had been telling his guardhouse *exactly* what a shape or a colour was. PW4 was the first time he let a model learn something from examples instead — and the first time he understood, viscerally, why data quality matters more than code quality in machine learning.

A. Objectives

1. Rapidly collect a dataset using Teachable Machine.
2. Train custom image and pose classification models using a visual UI.
3. Export the neural network to Visual Studio Code.
4. Build a touchless AI volume controller using hand tracking (pose estimation).

B. Learning Outcome

Use Teachable Machine to collect image datasets; apply image processing techniques to train custom image and pose classification models; and export the neural network model into VS Code to build a touchless AI volume controller.

 **Concept Bridge** — *Seeing Like a Machine* Unit 3.6 (Lab Blueprint D) walks through the theory of Teachable Machine's export/normalisation pipeline that Part D of this lab implements directly.

C. Topic Summary — Machine Learning in Computer Vision

This lab introduces the fundamentals of applying Machine Learning to Computer Vision, where computers are trained to interpret visual data such as images and video streams. You will apply **supervised learning** — the system learns from labelled datasets (e.g. “Mask On” and “No Mask”) to build an image classification model, then predicts new, unseen inputs by recognising the patterns and features a neural network extracted automatically.

The process begins with **data collection** — multiple images per class, captured with variation in lighting, angle, and position, to improve robustness. These feed the **training** phase, where a deep learning model learns the distinguishing features. Once trained, the model performs **inference** — making predictions on new, unseen data in real time from a webcam.

Data collection with Teachable Machine. This lab uses **Teachable Machine**, a no-code platform for rapid model development. The trained model exports in TensorFlow/Keras format (.h5) and deploys locally with Python: OpenCV captures live video, and TensorFlow runs it through the trained network to produce predictions with confidence scores. Before feeding images to the model, **preprocessing** is required — resizing (e.g. 224×224 pixels) and normalising pixel values to match the exact format used during training. Getting this preprocessing wrong is the single most common reason a Teachable Machine model “works in the browser but fails in Python.”

Pose estimation and hand tracking. Beyond raw images, this lab also introduces pose estimation, where the system detects human body positions rather than static pixels — classifying poses like “Hands Up” or “Hands Down” to demonstrate how ML interprets human actions, a technique widely used in gesture recognition, surveillance, and human-computer interaction. The final part uses **MediaPipe** for hand tracking, detecting 20 key landmarks on the hand. By identifying the thumb tip (landmark 4) and the index fingertip (landmark 8), the system calculates the Euclidean distance between them — and uses that distance as an input to control a system function, such as adjusting

computer volume, creating a fully touchless interaction system. The whole pipeline runs in real time: input acquisition – preprocessing – model inference – output visualisation.

D. Material / Tools

1. PC installed with Python 3.x and Visual Studio Code, plus a webcam
 2. Python libraries: NumPy, OpenCV (cv2)
 3. A face mask, or any household object (phone/pen/box) as an alternative pair of classes
 4. CIDOS Online Platform
-

E. General Instruction / Safety Procedure

See “Before every lab” in the introduction to this companion.

F. Work Instruction / Procedure

PART A — Data Collection and Training on Teachable Machine

1. Open <https://teachablemachine.withgoogle.com/> in your browser.
2. Click **Get Started** – **Image Project** – **Standard image model**.
3. **Define your classes:** rename Class 1 to “Mask On” (or any object you have handy, like “Phone”); rename Class 2 to “No Mask” (or a different object, like “Pen”).
4. **Collect data:** click the **Webcam** button for your first class, then hold **Hold to Record** to capture at least 200 images, moving your head/object slightly for varied angles and lighting. Repeat for the second class.
5. **Train and test:** click **Train Model** and do not switch tabs while training. Once done, test the model live in the browser preview.

PART B — Exporting the Neural Network

1. In the Preview panel, click **Export Model**.
2. Select the **TensorFlow** tab (not TensorFlow.js or TensorFlow Lite).
3. Choose the **Keras** model format, then **Download my model**.
4. Extract the downloaded .zip into a new project folder (e.g. Documents/CV_Lab4/). You should now have `keras_model.h5` (the trained network) and `labels.txt` (your class names).

PART C — Setting Up Your Local Environment

1. In VS Code, **File** – **Open Folder**, select your extracted CV_Lab4 folder.
2. Open a new terminal (**Terminal** – **New Terminal**) and confirm your virtual environment is active.

3. Install the required libraries:

Bash

```
pip install tensorflow opencv-python numpy    # skip any  
you already have  
pip install mediapipe pyautogui
```

4. If TensorFlow or Keras throws a version-mismatch error, check the installed versions:

Bash

```
pip show tensorflow  
pip show keras
```

and reinstall matching versions if needed:

Bash

```
pip install --upgrade pip  
pip install tensorflow==2.15.0 keras==2.15.0
```

PART D — Writing the Real-Time Inference Code

Create a new file named `webcam_inference.py`:

Python

```
# Task 1: Construct code to process from data model  
created by Teachable Machine  
# Objective: To generate inference of detection using  
model in local pc webcam using python  
import os  
import cv2  
import numpy as np  
from tensorflow.keras.models import load_model  
  
# Disable scientific notation for clarity  
np.set_printoptions(suppress=True)  
  
# 1. Load the model and the labels  
MODEL_PATH = "keras_model.h5"  
LABELS_PATH = "labels.txt"  
  
if not os.path.exists(MODEL_PATH) or not  
os.path.exists(LABELS_PATH):  
    raise FileNotFoundError(  
        "keras_model.h5 or labels.txt not found in this
```


Python

```
folder. "
    "Make sure you extracted your Teachable Machine
export here (Part B)."
)

print("Loading model...")
model = load_model(MODEL_PATH, compile=False)
class_names = open(LABELS_PATH, "r").readlines()
print("Model loaded successfully!")

# 2. Open the default webcam (Camera 0)
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise RuntimeError("Could not open the webcam. Check
that it is connected and not in use.")

print("Press 'q' to quit the program.")

while True:
    # 3. Read a frame from the camera
    ret, frame = cap.read()
    if not ret:
        break

    # 4. Preprocess the image to match the format
    Teachable Machine expects
    resized_frame = cv2.resize(frame, (224, 224),
interpolation=cv2.INTER_AREA)
    image_array = np.asarray(resized_frame,
dtype=np.float32).reshape(1, 224, 224, 3)
    normalized_image_array = (image_array / 127.5) - 1

    # 5. Run the prediction
    prediction = model.predict(normalized_image_array,
verbose=0)
    index = np.argmax(prediction)
    class_name = class_names[index].strip()
    confidence_score = prediction[0][index]

    # 6. Display the result on the screen
    text = f"{class_name}: {np.round(confidence_score *
100)}%"
    cv2.putText(frame, text, (10, 50),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
```

Python

```
cv2.imshow("Teachable Machine Local Inference",
frame)

# 7. Listen for the 'q' key to stop the loop
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

 **Fixed in this Companion** — Two robustness checks were added: (1) the script now confirms `keras_model.h5` and `labels.txt` actually exist in the folder before trying to load them, with a message pointing back to Part B if they're missing; and (2) it confirms the webcam actually opened (`cap.isOpened()`) before entering the capture loop, instead of silently looping on empty frames.

Capture the output video frame showing both **Mask On** and **No Mask** predictions, and discuss it in your report.

PART E — Pose Estimation with Teachable Machine

1. Go back to the Teachable Machine homepage and select **Pose Project**.
2. Record two physical poses: Class 1 “Hands Down” (stand normally), Class 2 “Hands Up” (raise both arms).
3. Train the model, export it as a **TensorFlow.js** model, and extract the files into a new folder (e.g. `CV_Lab4_Pose`).
4. Open this new folder in VS Code and create `pose_inference.py`.
5. Install the pose estimation library:

Bash

```
pip install mediapipe
```

6. Write or paste your pose inference script (available from the course repository) to see pose detection trigger an interactive alert.
7. Run `python pose_inference.py`.

PART F — AI Virtual Volume Controller

The goal: a touchless interface — pinch your thumb and index finger together to lower the computer's volume, spread them apart to raise it — built with **MediaPipe**, Google's fast hand/face/pose tracking framework.

Step 1 — the hand map. MediaPipe maps 21 landmarks (joints) on the hand. The thumb tip is landmark 4; the index fingertip is landmark 8.

Step 2 — the logic. Detect the hand → find the (x, y) of landmarks 4 and 8 → calculate the distance between them → trigger `pyautogui.press('volumeup')` if the distance is large, `pyautogui.press('volumedown')` if it's small.

Step 3 — the starter code. Create `volume_controller.py`:

Python

```
import cv2
import mediapipe as mp
import math
import pyautogui

# 1. Initialize MediaPipe Hands
mp_hands = mp.solutions.hands
hands = mp_hands.Hands(min_detection_confidence=0.7,
min_tracking_confidence=0.7)
mp_draw = mp.solutions.drawing_utils

cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise RuntimeError("Could not open the webcam. Check
that it is connected and not in use.")

print("Virtual Volume Controller Started! Press 'q' to
quit.")

while True:
    success, img = cap.read()
    if not success:
        break

    # Convert the BGR image to RGB (MediaPipe requires
    RGB)
    imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    results = hands.process(imgRGB)

    # If hands are detected on the screen
    if results.multi_hand_landmarks:
        for hand_landmarks in
results.multi_hand_landmarks:
            mp_draw.draw_landmarks(img, hand_landmarks,
mp_hands.HAND_CONNECTIONS)
```

Python

```
# Get the coordinates for Thumb Tip (4) and
Index Finger Tip (8)
lm_list = []
h, w, c = img.shape
for id, lm in
enumerate(hand_landmarks.landmark):
    cx, cy = int(lm.x * w), int(lm.y * h)
    lm_list.append([id, cx, cy])

    if len(lm_list) != 0:
        x1, y1 = lm_list[4][1], lm_list[4][2]
        x2, y2 = lm_list[8][1], lm_list[8][2]

        # Draw circles on the tips and a line
between them
        cv2.circle(img, (x1, y1), 10, (255, 0,
255), cv2.FILLED)
        cv2.circle(img, (x2, y2), 10, (255, 0,
255), cv2.FILLED)
        cv2.line(img, (x1, y1), (x2, y2), (255,
0, 255), 3)

        # Calculate distance
        length = math.hypot(x2 - x1, y2 - y1)
        print(f"Finger Distance: {int(length)}")

        #
=====
        # YOUR CHALLENGE STARTS HERE
        #
=====
        # Hint 1: Find out what 'length' is when
your fingers are pinched,
        # and what it is when they are spread
wide.

        # Hint 2: Use an 'if' statement! Example:
        # if length > 150:
        #     pyautogui.press('volumeup')
        # elif length < 50:
        #     pyautogui.press('volumedown')

        # Write your volume control logic here!

cv2.imshow("AI Volume Controller", img)
```

Python

```
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

Try filling in the volume logic yourself first — pinch and spread your fingers a few times while watching the printed `Finger Distance` values to find your own thresholds. If you get stuck, one reasonable solution (using a short cooldown so the volume doesn't change on every single frame) looks like this:

Python

```
# One possible solution – a simple
threshold with a cooldown
import time
if "last_action_time" not in dir():
    last_action_time = 0

now = time.time()
if now - last_action_time > 0.3: #
cooldown so volume doesn't spam-change
    if length > 150:
        pyautogui.press('volumeup')
        last_action_time = now
    elif length < 50:
        pyautogui.press('volumedown')
        last_action_time = now
```

⚠ Common Mistake — Without a cooldown, `pyautogui.press()` fires on *every single frame* your fingers stay past the threshold — at 30 frames per second that's 30 volume changes a second. A short cooldown (or checking that the volume actually needs to change) keeps the control usable.

G–J. Result, Discussion, Conclusion, References

Follow the reporting structure in this companion's introduction. References: OpenCV Tutorial (Great Learning), Programming for Everybody (Coursera), and Ansari (2023) — *Building Computer Vision Applications Using Artificial Neural Networks* (2nd ed.), Apress.

PRACTICAL WORK 5 — Object, Face, and Pose Detection with YOLOv8 via VS Code and Google Colab

CLO 2 · Construct suitable application using machine learning method · **PL04, P4** · DK3, DK4, DK8 / DP1, DP4

Amir's Notebook

By the final lab, Amir's guardhouse had grown too demanding for his laptop's CPU — a full YOLOv8 model detecting people, vehicles, and poses in real time needed a GPU he didn't own. PW5 solves that with a trick most working engineers use daily: borrow someone else's GPU over the internet, and code against it as if it were sitting on your own desk.

A. Objectives

1. Establish a remote SSH-style connection between Visual Studio Code and a cloud-based GPU.
2. Run professional object detection scripts to extract data analytics.
3. Manipulate bounding box coordinates to create a facial privacy filter.
4. Track human poses in a video feed.

B. Learning Outcome

Use Google Colab to provision GPU processing; apply image processing techniques with a custom image and pose model; and export results back to your local computer via VS Code to build human tracking in a video feed.

 **Concept Bridge** — *Seeing Like a Machine* Unit 3.6 (Lab Blueprint E) and Unit 4.3 cover the theory of training and running YOLOv8; this lab is where you actually run it against a live GPU.

C. Topic Summary — YOLOv8 and Google Colab GPUs

YOLOv8 (You Only Look Once, version 8) is a state-of-the-art real-time object detection model and computer vision framework, developed by Ultralytics and released in January 2023. It handles object detection, instance segmentation, pose estimation, tracking, and image classification within one unified framework.

Compared with earlier YOLO versions, YOLOv8 introduces **anchor-free detection** (predicting an object's centre directly, rather than relying on predefined anchor boxes, which speeds up non-maximum suppression and simplifies training), a new **CSPDarknet53 backbone with a C2f module** (improving feature extraction and gradient flow without much added computational cost), a **decoupled head** (separating objectness, classification, and regression into independent branches for faster convergence),

and a **unified API** across detection, segmentation, and classification. Its Nano variant (YOLOv8n) achieves 37.3 mAP on COCO at 0.99 ms on an A100 GPU — a strong accuracy/speed tradeoff.

Unlike sliding-window detectors, YOLO processes the entire image at once, which is what makes it fast enough for real-time video. Thanks to multi-scale feature maps, it can detect small objects and objects in crowded scenes. It ships in five sizes trading speed against accuracy: **Nano** (fastest, ideal for edge devices), **Small**, **Medium**, **Large**, and **Extra Large** (most accurate, slowest). Typical use cases span surveillance and security, autonomous vehicles, industrial inspection, agriculture, and medical imaging.

How YOLO works, mechanically: the input image is resized for consistency; the image is divided into a grid, where each cell is responsible for detecting objects whose centre falls inside it; a convolutional backbone extracts features; and each grid cell predicts bounding boxes (x, y, width, height, confidence) plus class probabilities.

YOLOv7 vs MediaPipe for pose, for context: YOLO-family pose models use the 17-keypoint COCO topology, run detection on every frame, support both CPU and GPU, and handle multiple people at once; MediaPipe Pose uses a 33-keypoint topology (COCO plus palm and face detail), runs detection once and then tracks until an occlusion forces re-detection, is CPU-only, has segmentation built in, and is designed for a single person.

Google Colab is a free, cloud-based, Jupyter-based platform giving browser access to GPUs (like the NVIDIA T4) without any local setup, with TensorFlow and PyTorch pre-installed. Free sessions run up to roughly 12 hours before the virtual machine resets and unsaved local data is lost; files saved directly to the notebook's file browser are temporary, so mount Google Drive if you need to keep anything permanently; and GPU access is shared, so turn the runtime off when you're not actively using it.

D. Material / Tools

1. PC installed with Python 3.x and Visual Studio Code, plus a webcam
2. Python libraries: NumPy, OpenCV (cv2)
3. CIDOS Online Platform
4. A Google account
5. A free GitHub account

E. General Instruction / Safety Procedure

See “Before every lab” in the introduction to this companion.

F. Work Instruction / Procedure

PART A — Connecting VS Code to a Google Colab GPU (Mandatory Setup)

Professional developers prefer a full IDE like VS Code over a browser tab. This part connects your local VS Code directly to the GPU machine behind a Colab session, using **VS Code Remote Tunnels** — the officially supported integration from Microsoft and Google. Once connected, the remote VS Code window

(file explorer and terminal included) runs *on* the Colab machine, not your laptop — every command in Parts B–F below executes on Colab’s free GPU.

Older guides used a raw SSH tunnel (`colab -ssh` + Cloudflare). Google now restricts direct SSH access to Colab, and that approach can flag your account. Remote Tunnels is the supported replacement — more stable, no Cloudflare account needed, and a real terminal on the GPU. You only need a free GitHub account.

Step A.1 — Install the Remote Tunnels extension. In VS Code, open Extensions (`Ctrl+Shift+X`/`Cmd+Shift+X`), search for **Remote - Tunnels** (published by Microsoft), and install it. (Already covered if you have the “Remote Development” extension pack.)

Step A.2 — Confirm you have a GitHub account. The tunnel authorises through GitHub. Create a free account at <https://github.com/signup> if needed — you must use the **same** GitHub account on both Colab and VS Code.

Step A.3 — Open Google Colab and turn on the GPU. 1. Open a new notebook at <https://colab.research.google.com/>. 2. **Critical:** *Runtime* → *Change runtime type* → set Hardware accelerator to **T4 GPU** → **Save**. 3. (Optional check) Run `!nvidia-smi` in a cell — a Tesla T4 should be listed.

Step A.4 — Start the VS Code tunnel from Colab. In the first code cell:

Python

```
# Install the helper that opens a VS Code Remote Tunnel  
on the Colab machine  
!pip install vscode-colab --upgrade  
  
import vscode_colab  
  
# Authenticate this Colab runtime with your GitHub  
account  
vscode_colab.login()
```

Run the cell — Colab prints a device code and a link (<https://github.com/login/device>). Open the link, enter the code, and authorise (once per session). Then, in the next cell:

Python

```
# Open the tunnel and give the Colab machine a name you  
can connect to  
vscode_colab.connect(name="cv-lab5")
```

 **Common Mistake** — Closing the Colab tab or stopping this cell ends the connection immediately and disconnects VS Code. Keep this cell running and the Colab tab open for the entire lab.

Step A.5 — Connect your local VS Code to the tunnel. 1. Open the Command Palette (**Ctrl+Shift+P**/**Cmd+Shift+P**). 2. Select **Remote Tunnels: Connect to Tunnel...** 3. Choose **GitHub** and sign in with the same account used in Colab. 4. Select the tunnel named **cv-lab5**. A new VS Code window opens; the green status bar bottom-left should read **Tunnel: cv-lab5**.

Step A.6 — Open the working folder and verify the GPU. 1. *Open Folder* → type `/content` → confirm (Colab's working directory). 2. New terminal: *Terminal* → *New Terminal*. 3. Create your lab folder: `bash mkdir -p /content/CV_Lab5 cd /content/CV_Lab5` 4. Confirm the GPU: `bash nvidia-smi` You should see an NVIDIA Tesla T4 listed. Run every command in Parts B–F in this same terminal, inside `/content/CV_Lab5`, with the Colab tab kept open.

PART B — Installing YOLOv8 and Workspace Setup

Bash

```
pip install ultralytics
```

Verify the install and confirm the GPU is detected:

Bash

```
yolo checks
```

The output should report the Ultralytics version and confirm CUDA (the T4 GPU) is available. YOLO automatically downloads pretrained weights (e.g. `yolov8n.pt`) the first time a script requests them.

PART C — Object Detection & Data Analytics

Instead of just eyeballing an output image, extract real numbers.

Bash

```
wget https://ultralytics.com/images/bus.jpg -O street.jpg
```

Create `analytics.py`:

Python

```
from ultralytics import YOLO
import cv2
import numpy as np

# Load the base YOLOv8 model (weights download
# automatically on first run)
model = YOLO('yolov8n.pt')

# Run inference on the image
results = model('street.jpg')
```

Python

```
# Extract data: YOLO stores detected class IDs in
results[0].boxes.cls
detected_classes = results[0].boxes.cls.cpu().numpy()

# In the COCO dataset, class 0 = 'person' and class 5 =
'bus'
person_count = int(np.sum(detected_classes == 0))
bus_count = int(np.sum(detected_classes == 5))
print(f"Analytics Report: Found {person_count} people and
{bus_count} bus(es).")

# Optional: print every detected object with its
confidence score
for box in results[0].boxes:
    class_id = int(box.cls[0])
    conf = float(box.conf[0])
    print(f" - {model.names[class_id]} ({conf:.2f})")

# Save the annotated image so you can view it
res_image = results[0].plot()
cv2.imwrite("street_analytics.jpg", res_image)
print("Saved annotated image as street_analytics.jpg")
```

Run `python analytics.py`, then open `street_analytics.jpg` in the Explorer to view the result.

PART D — Face Detection & Privacy Blurring

Detect people with YOLO, read each bounding box, and use OpenCV to blur the top region (approximating the head/face) — a realistic surveillance-anonymisation pipeline.

Create `privacy_blur.py`:

Python

```
from ultralytics import YOLO
import cv2

# Load the model and the image
model = YOLO('yolov8n.pt')
results = model('street.jpg')
img = cv2.imread('street.jpg')
h, w = img.shape[:2]

# Loop through every detected bounding box
for box in results[0].boxes:
```

Python

```
class_id = int(box.cls[0])
# Only act on people (class_id == 0 in COCO)
if class_id == 0:
    # Bounding box coordinates [x1, y1, x2, y2]
    x1, y1, x2, y2 = map(int, box.xyxy[0])
    # Clamp coordinates so they stay inside the image
    x1, x2 = max(0, x1), min(w, x2)
    y1, y2 = max(0, y1), min(h, y2)

    # Approximate the face: top 20% of the person's
box
    face_y2 = y1 + int((y2 - y1) * 0.2)
    face_region = img[y1:face_y2, x1:x2]

    # Skip empty regions to avoid OpenCV errors
    if face_region.size == 0:
        continue

    # Heavy Gaussian blur on the face region
    blurred_face = cv2.GaussianBlur(face_region, (51,
51), 0)
    img[y1:face_y2, x1:x2] = blurred_face

# Save the anonymised image
cv2.imwrite("street_anonymized.jpg", img)
print("Privacy filter applied and saved as
street_anonymized.jpg")
```

Run `python privacy_blur.py` and inspect the result. The coordinates are already clamped to the image bounds and empty regions are skipped, so the script won't crash when a person is detected right at the edge of the frame.

PART E — Human Pose Estimation on Video

Bash

wget

```
https://github.com/intel-iot-devkit/sample-videos/raw/master/people-walking.mp4 -O walking.mp4
```

Create `pose.py`:

Python

```
from ultralytics import YOLO
```

Python

```
# Load the YOLOv8 Pose model
pose_model = YOLO('yolov8n-pose.pt')

# Run inference on the video and save the annotated
output
print("Processing video... this may take a minute.")
pose_model.predict(
    source='walking.mp4',
    save=True,
    project='runs/pose',
    name='pose_output',
    exist_ok=True # overwrite instead of creating
pose_output2, 3, ...
)
print("Done! Check the runs/pose/pose_output directory.")
```

Run `python pose.py`. Open `runs/pose/pose_output` in the Explorer and download the output video to your local machine to watch the skeletal tracking. The fixed `project/name/exist_ok` combination means re-running the script overwrites the same folder instead of littering `pose_output2`, `pose_output3`, and so on.

PART F — Final Lab Challenge

Build your own pipeline in a script named `challenge.py` that:

1. Downloads an image of a crowded street or highway.
2. Runs the object detection model on it.
3. Prints an “Analytics Report” counting the total number of vehicles (cars, trucks, buses, motorcycles).
4. Applies a solid black rectangle over every detected vehicle with OpenCV, completely blocking them out.
5. Saves the final image as `challenge_complete.jpg`.

Hint — COCO vehicle class IDs: car = 2, motorcycle = 3, bus = 5, truck = 7. To draw a solid box: `cv2.rectangle(img, (x1, y1), (x2, y2), (0, 0, 0), -1)` — the `-1` thickness fills the rectangle.

Try building this yourself first using the patterns from Parts C and D. If you’d like a reference point once you’ve made a genuine attempt, one working approach looks like this:

Python

```
# One possible solution for the Final Lab Challenge
from ultralytics import YOLO
import cv2
```

Python

```
VEHICLE_CLASS_IDS = {2, 3, 5, 7}    # car, motorcycle,
bus, truck

model = YOLO('yolov8n.pt')
results = model('highway.jpg')      # your downloaded
crowded street/highway image
img = cv2.imread('highway.jpg')

vehicle_count = 0
for box in results[0].boxes:
    class_id = int(box.cls[0])
    if class_id in VEHICLE_CLASS_IDS:
        vehicle_count += 1
        x1, y1, x2, y2 = map(int, box.xyxy[0])
        cv2.rectangle(img, (x1, y1), (x2, y2), (0, 0, 0),
-1)

print(f"Analytics Report: Found {vehicle_count}
vehicle(s).")
cv2.imwrite("challenge_complete.jpg", img)
print("Saved challenge_complete.jpg")
```

G. Result

Show all the code, with a comment for each task, in your report (Parts C, D, E, and F). Include the output of each script — the analytics report text and the saved images/video (`street_analytics.jpg`, `street_anonymized.jpg`, the pose output video, and `challenge_complete.jpg`). Upload the code to CIDOS and print the report.

H. Discussion

Discuss what you observed. For example: how many objects did the model detect, and were any missed or mislabelled? How well did the privacy blur cover faces, and what happens when people overlap? How did processing on the Colab GPU compare with running on a CPU? What are the limitations of using an approximate “top 20%” region for the face?

I. Conclusion

Summarise whether your findings supported the objectives, identify possible sources of error (low-confidence detections, occlusion, approximate face regions), and state what you learned from connecting VS Code to a cloud GPU and running object, face, and pose detection with YOLOv8.

J. References

1. OpenCV Tutorial: A Guide to Learn OpenCV in Python — Great Learning, <https://www.mygreatlearning.com/blog/opencv-tutorial-in-python/>
 2. Ansari, S. (2023). *Building Computer Vision Applications Using Artificial Neural Networks* (2nd ed.). Apress.
 3. <https://www.v7labs.com/blog/yolo-object-detection>
 4. <https://learnopencv.com/yolov7-pose-vs-mediapipe-in-human-pose-estimation/>
 5. Ultralytics YOLOv8 documentation: <https://docs.ultralytics.com/>
 6. VS Code Remote Tunnels: <https://code.visualstudio.com/docs/remote/tunnels>
 7. vscode-colab (PyPI): <https://pypi.org/project/vscode-colab/>
-

Appendix — Map to *Seeing Like a Machine*

Practical Work	Companion Focus	Matching Unit in <i>Seeing Like a Machine</i>
PW1 — Introduction to Python Programming	Environment setup, dynamic typing, control flow, data structures, OOP detector paradigm	Unit 1 (Foundations of Python Programming)
PW2 — Python Basics for Image Processing	NumPy image representation, grayscale, resizing, ImageProcessor OOP class	Unit 1.4 (OOP) and Unit 2.1–2.2 (Coordinate system, colour spaces)
PW3 — OpenCV Image and Video Processing	Colour spaces, masking, blurring, Canny, contours, tracking, OCR, QR/barcode	Unit 2 in full (all Lab Blueprints A–C)
PW4 — Model Training with Teachable Machine & TensorFlow	Supervised learning workflow, no-code model training, MediaPipe hand tracking	Unit 3 (Terminology, Workflow, Tools) — Lab Blueprint D
PW5 — YOLOv8 via VS Code and Google Colab	Cloud GPU workflow, YOLOv8 object/face/pose detection, analytics, privacy filtering	Unit 3.5–3.6 (Frameworks, Lab Blueprint E) and Unit 4 (Applied Projects)

End of companion volume.